PROFILEAUDITOR
AI-Powered Resume Claim Verification & Reality Scoring System
Comprehensive Laboratory Report (Experiments 6 to 12)

Course: Software Engineering & DevOps Lab
GitHub Repository: https://github.com/AdityaSinghal28/ProfileAuditor
Upstream Repository: https://github.com/vishvjeettanwar1623/ProfileAuditor
Date: November 2026

==
EXECUTIVE SUMMARY
==
This laboratory report details the design, implementation, component integration,
automated unit testing, version control branching model, continuous integration (CI)
pipeline, IEEE 829 test planning, and final deployment documentation for Profile Auditor.
The experiments cover the complete software engineering lifecycle from Experiment 6
to Experiment 12.

ProfileAuditor resolves candidate credential inflation in modern recruitment by
extracting claims from resumes (PDF/DOCX) and cross-referencing them in real-time
against public developer footprints (GitHub repositories, commits, stars, languages,
and LeetCode problem-solving activity). It computes an empirical Reality Score (0-100)
and an ATS Health Score to corroborate technical authenticity.

==
EXPERIMENT 6: IMPLEMENTATION OF CORE MODULES USING A SUITABLE TECHNOLOGY STACK
==
1. OBJECTIVE:
To architect and implement core backend modules, document ingestion parsers, footprint
auditing services, and domain data models using Python 3 and FastAPI.

2. ARCHITECTURAL DECISIONS & TECH STACK:
- Backend: FastAPI (Python 3.11/3.13) for high-performance asynchronous async/await execution,
  Pydantic request validation, and auto-generated OpenAPI (Swagger) documentation.
- Document Ingestion: Multi-tier fallback architecture:
  * PyMuPDF (fitz) for high-speed binary stream parsing.
  * pdfminer.six for structural text and layout extraction.
  * pdfplumber for tabular layout extraction.
  * python-docx and mammoth for Word .docx document parsing.
- Public Footprint Verification:
  * GitHub REST API v3 for repositories, stars, languages, and commit frequency.
  * LeetCode GraphQL public endpoint for algorithmic problem-solving metrics.
- Decoupled Storage Layer:
  * app.core.storage: Thread-safe singleton with automated TTL expiration pruning,
    ensuring zero database dependency for stateless deployment.

3. CORE MODULE RESPONSIBILITIES:
- app.core.storage: In-memory store lifecycle with automated TTL eviction.
- app.services.resume_parser: Extracts text, emails, phones, GitHub and LeetCode handles.
- app.services.github_service: Queries GitHub, aggregates stars, and runs fuzzy

project matching.
- app.services.leetcode_service: Verifies algorithmic tiers (Easy, Medium, Hard)
.
- app.services.scoring_service: Synthesizes weighted composite Reality Score (0-
100).
- app.services.ats_service: Audits action verb density and quantifiable metric c
ounts.

4. OUTCOME & VERIFICATION:
All modules compile cleanly without circular dependencies. Isolated imports and
parsing tests executed successfully.

==
EXPERIMENT 7: CONTINUED IMPLEMENTATION OF CORE MODULES; COMPONENT INTEGRATION
==
1. OBJECTIVE:
To integrate individual backend modules into a unified RESTful service and conne
ct
the React frontend client with the FastAPI backend API pipeline.

2. END-TO-END DATA FLOW:
1. Candidate uploads resume (PDF/DOCX) via React drag-and-drop.
2. React dispatches multipart/form-data to POST /api/resume/upload.
3. Backend parses resume, extracts handles and skills, stores intermediate state
 in store.
4. Frontend triggers POST /api/verification/verify/{resume_id}.
5. Backend executes parallel audits against GitHub and LeetCode.
6. Scoring engine computes Reality Score (0-100) and ATS Health Score.
7. Frontend retrieves final score via GET /api/score/{resume_id} and renders rad
ar charts.

3. CORS & SECURITY CONFIGURATION:
Configured CORSMiddleware in app/main.py allowing http://localhost:3000, http://
127.0.0.1:3000,
and production endpoints with credentials and wildcard headers.

4. LIVE INTEGRATION VERIFICATION:
- GET http://localhost:8000/ -> HTTP 200 OK {"message":"ProfileAuditor Backend A
PI","status":"running"}
- GET http://localhost:8000/health -> HTTP 200 OK {"status":"ok"}
- GET http://localhost:8000/docs -> HTTP 200 OK (Swagger OpenAPI UI rendered)
- GET http://localhost:3000/ -> HTTP 200 OK (React Dashboard active)

==
EXPERIMENT 8: WRITING UNIT TESTS USING APPROPRIATE FRAMEWORKS (PyTest / Jest)
==
1. OBJECTIVE:
To construct and execute comprehensive unit test suites and integration verifica
tion
suites using PyTest (backend) and Jest (frontend).

2. TEST SUITES IMPLEMENTATION:
- Backend:
  * pytest.ini configured for automated discovery.
  * test_storage_and_api.py: Asserts store set/get, TTL pruning, root/health han
dlers,
    and live ATS scoring endpoint.
  * test_project_matching.py: Validates fuzzy matching of candidate resume proje
cts
    against varied GitHub repository names (e.g. 'Questfi' -> 'Questfi-Vietbuild
').
  * test_comprehensive.py: Full multi-candidate end-to-end pipeline (Senior Engi

neer,
    Student, DevOps Engineer).
  * run_all_tests.py: Master runner script executing all test suites sequentiall
y.
- Frontend:
  * frontend/src/utils/score.test.js: Jest unit tests asserting score threshold
    categorization (toneForPercent) and semantic metadata labels (getScoreMeta).

3. TEST EXECUTION RESULTS:
- Backend test_storage_and_api.py: PASSED (2/2 tests) in 0.51s
- Backend test_project_matching.py: PASSED (1/1 test) in 0.44s
- Backend test_comprehensive.py: PASSED (7/7 tests) in 4.20s
- Frontend score.test.js: PASSED (4/4 tests) in 0.61s
Overall Test Pass Rate: 100.0% (Zero test failures).

==
EXPERIMENT 9: SETUP OF GIT REPOSITORY; BRANCHING MODEL; PR & CODE REVIEW PROCESS
==
1. OBJECTIVE:
To establish structured version control, enforce a Git Flow branching model, and
configure standardized Pull Request templates and code review practices on GitHu
b.

2. REPOSITORY TOPOLOGY:
- Origin: https://github.com/AdityaSinghal28/ProfileAuditor.git
- Upstream: https://github.com/vishvjeettanwar1623/ProfileAuditor.git

3. BRANCHING STRATEGY (Git Flow):
- main: Production-ready release branch protected against unreviewed commits.
- develop: Continuous integration branch where all feature branches converge.
- feature/experiments-implementation: Dedicated topic branch for PyTest, Jest, C
I,
  and documentation enhancements.

4. PULL REQUEST GOVERNANCE:
Engineered .github/pull_request_template.md requiring:
- Description and issue linking.
- Change classification (bug fix, new feature, breaking change).
- Verification checklist (local tests, CI checks, documentation updates).
- Reviewer self-assessment criteria.

5. EXECUTION & SYNCHRONIZATION:
All branches were merged using non-fast-forward merge strategy and synchronized:
  git push origin main develop feature/experiments-implementation

==
EXPERIMENT 10: CONFIGURATION OF CONTINUOUS INTEGRATION (CI) USING GITHUB ACTIONS
==
1. OBJECTIVE:
To design and automate a Continuous Integration pipeline using GitHub Actions to
enforce build and test verification on every commit and pull request.

2. CI PIPELINE SPECIFICATION (.github/workflows/ci.yml):
- Triggers: Push and Pull Request events to 'main' and 'develop' branches.
- Job 1: backend-test (ubuntu-latest):
  * Sets up Python 3.11 with pip caching.
  * Installs backend requirements.
  * Runs PyTest suites (pytest tests/test_storage_and_api.py tests/test_project_
matching.py).
  * Runs master test runner (python tests/run_all_tests.py).
- Job 2: frontend-test-and-build (ubuntu-latest):
  * Sets up Node.js 18.x with npm caching.

  * Clean install: npm ci.
  * Executes Jest unit tests: CI=true npm test.
  * Executes production build: npm run build.

3. LIVE CI EXECUTION VERIFICATION:
- Workflow Run ID: 43175325525
- Trigger: push (branch: main)
- Backend Job: PASSED in 32s (Job ID: 131038068875)
- Frontend Job: PASSED in 54s (Job ID: 131038069119)
- Total Duration: 57s
- Overall Conclusion: SUCCESS (Green checkmark)
- Audit URL: https://github.com/AdityaSinghal28/ProfileAuditor/actions/runs/4317
5325525

==
EXPERIMENT 11: PREPARATION OF TEST PLAN, TEST CASES, AND TEST REPORTS
==
1. OBJECTIVE:
To develop an IEEE 829 Software Test Plan, execute exhaustive functional, bounda
ry,
and edge test cases, and assemble an empirical test summary report.

2. IEEE 829 SOFTWARE TEST PLAN (STP):
- Scope: ProfileAuditor resume parser, verification engine, scoring algorithms,
  and React dashboard.
- Approach: Combined White-Box unit test coverage with Black-Box boundary testin
g.
- Environment: Python 3.11/3.13, Node 18+, Chrome 120+, Ubuntu 22.04 LTS / macOS
.

3. STRUCTURED TEST CASES EXECUTION:
TC-01: Storage set/get lifecycle   -> PASSED
TC-02: Storage TTL ancient pruning   -> PASSED (1 old entry pruned)
TC-03: GitHub handle regex extraction   -> PASSED ('alexchen' extracted)
TC-04: LeetCode handle regex extraction   -> PASSED ('alexchen' extracted)
TC-05: Malformed document upload handling   -> PASSED (HTTP 400 graceful catch)
TC-06: Exact project-to-repo matching   -> PASSED (Corroborated = True)
TC-07: Fuzzy project-to-repo matching   -> PASSED (Corroborated = True)
TC-08: Unverified project claim handling   -> PASSED (Corroborated = False)
TC-09: Reality Score >= 80 semantic mapping -> PASSED (tone: 'ok', Strong label
)
TC-10: Reality Score = 65 semantic mapping   -> PASSED (tone: 'warn', Moderate l
abel)
TC-11: Reality Score = 30 semantic mapping   -> PASSED (tone: 'bad', Low label)
TC-12: Backend Root endpoint status check   -> PASSED (HTTP 200, status: runnin
g)
TC-13: Backend Health endpoint probe   -> PASSED (HTTP 200, status: ok)
TC-14: Frontend Webpack production build   -> PASSED (Compiled successfully)
TC-15: GitHub Actions Automated CI Gate   -> PASSED (All 2 matrix jobs green)

4. TEST SUMMARY METRICS:
- Total Cases: 15
- Passed: 15 (100.0%)
- Failed: 0 (0.0%)
- Defect Density: 0 critical defects.

==
EXPERIMENT 12: FINAL PROJECT IMPLEMENTATION, DEBUGGING, DOCUMENTATION & MANUAL
==
1. OBJECTIVE:
To consolidate the complete ProfileAuditor solution, conduct system debugging an
d

refactoring, assemble complete API and system manuals, and prepare the final rep
ort.

2. DEBUGGING & REFACTORING LOG:
- Issue: PyTest suite discovery omitted.
  Solution: Added backend/pytest.ini, wrapped tests in test_* functions.
- Issue: Missing 'test' script in frontend/package.json.
  Solution: Added react-scripts test script and authored score.test.js.
- Issue: In-memory storage coupling in route files.
  Solution: Refactored state into app.core.storage singleton with automated TTL
pruning.
- Issue: CORS rejection on cross-port access.
  Solution: Added explicit origin whitelist in app/main.py for localhost:3000.

3. SYSTEM API SPECIFICATION REFERENCE:
- POST /api/resume/upload: Uploads resume (PDF/DOCX) and returns resume_id + par
sed claims.
- GET /api/resume/{id}: Retrieves stored parsed resume claims.
- POST /api/verification/verify/{id}: Initiates async GitHub & LeetCode verifica
tion.
- GET /api/score/{id}: Returns Reality Score (0-100) and verified badges.
- POST /api/score/live-ats: Evaluates live text for ATS formatting, action verbs
, metrics.
- GET /health: Microservice health check probe.

4. LOCAL & DOCKER DEPLOYMENT PROCEDURES:
- Local Backend:
  cd backend && python -m venv .venv && source .venv/bin/activate
  pip install -r requirements.txt
  python -m uvicorn app.main:app --port 8000 --reload
- Local Frontend:
  cd frontend && npm install && npm start
- Docker Compose:
  docker-compose up --build -d

5. CONCLUSION:
Experiments 6 through 12 have been executed and validated against the ProfileAud
itor
codebase. The system features modular architecture, high test coverage, robust G
it
governance, and an automated continuous integration pipeline.